COMPUTER ARCHITECTURE & ORGANIZATION

TWO MARKS QUESTIONS AND ANSWERS

MODULE 1

DIGITAL COMPUTERS, REGISTER TRANSFER LANGUAGE

1 . Define Computer Organization.

Computer Organization refers to the operational structure and functional behavior of a computer system. It focuses on how the various components of a computer (such as the CPU, memory, input/output devices, and storage) are connected and interact to execute instructions.

2 . Define Computer Architecture.

Computer Architecture is the design and structure of a computer system that defines how it operates, including its instruction set, data formats, and methods for data processing, storage, and communication. It focuses on the system's functionality as seen by programmers.

3 . Define Computer Design.

Computer Design refers to the process of creating the physical hardware components of a computer system based on its architecture. It involves selecting and interconnecting components like the CPU, memory, and I/O devices to implement the desired functionality efficiently.

4 . Define Register Transfer Language (RTL).

Register Transfer Language (RTL) is a symbolic language used to describe the operations, data transfers, and control actions between registers in a digital system. It is commonly used in designing and modeling computer hardware at the micro-operation level.

5 . Define Microoperation

A microoperation is a basic operation performed on data stored in registers, such as transferring data, performing arithmetic, or logic functions. It is the smallest unit of operation in a computer's control unit.

6 . List out Arithmetic microoperations.

TABLE 4-3 Arithmetic Microoperations

Symbolic designation	Description
$R3 \leftarrow R1 + R2$	Contents of $R1$ plus $R2$ transferred to $R3$
$R3 \leftarrow R1 - R2$	Contents of $R1$ minus $R2$ transferred to $R3$
$R2 \leftarrow \overline{R2}$	Complement the contents of $R2$ (1's complement)
$R2 \leftarrow \overline{R2} + 1$	2's complement the contents of $R2$ (negate)
$R3 \leftarrow R1 + \overline{R2} + 1$	$R1$ plus the 2's complement of $R2$ (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of $R1$ by one
$R1 \leftarrow R1 - 1$	Decrement the contents of $R1$ by one

7 . List out Logical micro operations.

Output	Operation
$E = A \wedge B$	AND
$E = A \vee B$	OR
$E = A \oplus B$	XOR
$E = \overline{A}$	Complement

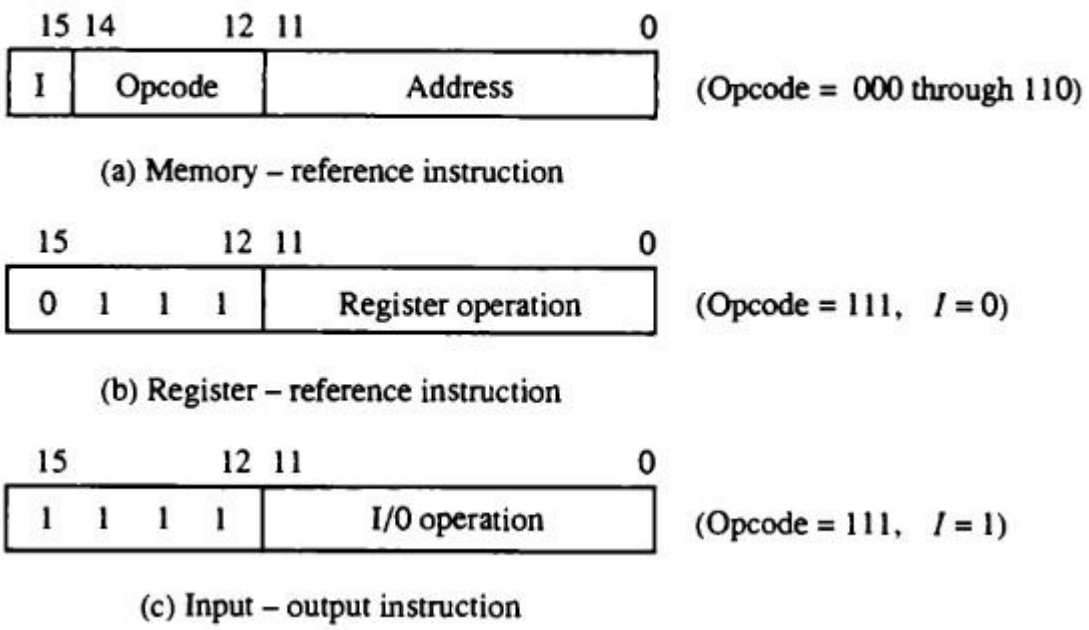
8 . Describe Instruction format.

An instruction format is the binary layout of an instruction in memory. It specifies how an instruction is divided into fields that tell the CPU what operation to perform and where to find the operands.. It is divided into fields such as:

- Opcode – Specifies the operation to perform
- Operand(s) – Specifies the data or address to be used
- Mode field (optional) – Indicates the addressing mode.

9 . List three types of basic Instruction format

Figure 5-5 Basic computer instruction formats.



10 . List out Memory reference Instructions

Mnemonic	Instruction	Operation (Micro-operation)	Description
AND	AND Address	AC ← AC ∧ M[Address]	Logical AND between AC and memory word.
ADD	ADD Address	AC ← AC + M[Address]	Adds memory word to AC.
LDA	LDA Address	AC ← M[Address]	Loads operand from memory into AC.
STA	STA Address	M[Address] ← AC	Stores content of AC into memory.
BUN	BUN Address	PC ← Address	Branch unconditionally to the given address.
BSA	BSA Address	M[Address] ← PC, PC ← Address+1	Branch and save return address in memory.
ISZ	ISZ Address	M[Address] ← M[Address] + 1; if 0 then PC ← PC+1	Increments memory word; skips next instruction if result is zero.

11 . List out Register reference Instructions

Mnemonic	Instruction	Operation (Micro-operation)	Description
CLA	Clear AC	AC ← 0	Clears the accumulator.
CLE	Clear E	E ← 0	Clears the E (carry/extend) flip-flop.
CMA	Complement AC	AC ← AC'	Complements (1's complement) accumulator contents.
CME	Complement E	E ← E'	Complements the E flip-flop.
CIR	Circulate Right	AC(0) ← E, E ← AC(15), AC ← shr AC	Circular right shift of AC and E.
CIL	Circulate Left	AC(15) ← E, E ← AC(0), AC ← shl AC	Circular left shift of AC and E.
INC	Increment AC	AC ← AC + 1	Increments accumulator by 1.
SPA	Skip if Positive AC	If (AC(15) = 0) → PC ← PC + 1	Skips next instruction if AC is positive.
SNA	Skip if Negative AC	If (AC(15) = 1) → PC ← PC + 1	Skips next instruction if AC is negative.
SZA	Skip if AC = 0	If (AC = 0) → PC ← PC + 1	Skips next instruction if AC is zero.

SZE	Skip if E = 0	If (E = 0) → PC ← PC + 1	Skips next instruction if E flip-flop is zero.
HLT	Halt	S ← 0	Stops the computer.

12 . List out I/O reference Instructions

Some common I/O Reference Instructions are:

- IN – Read data from an input device
- OUT – Send data to an output device
- GET – Receive data from an input device (similar to IN)
- PUT – Send data to an output device (similar to OUT)
- TEST – Check status of an I/O device.

13 . List out Computer Registers.

TABLE 5-1 List of Registers for the Basic Computer

Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

14 . Define Interrupt.

An Interrupt is a signal sent to the CPU that temporarily halts the current program execution to give attention to a more urgent task or event. It allows the CPU to respond quickly to important events like input/output requests, hardware malfunctions, or timers. After handling the interrupt, the CPU resumes the interrupted program.

15 . Define Bus.

A bus is a set of wires that transfers data, addresses, and control signals between different parts of a computer.

Module 2: Microprogrammed Control and Central Processing Unit

1 . Define Control unit.

The Control Unit (CU) is a part of the CPU that directs and coordinates the operations of the computer by interpreting instructions and generating control signals to manage data flow and execute tasks.

2 . List the types of control unit.

- Hardwired Control Unit
- Microprogrammed Control Unit

These differ in how they generate control signals to manage CPU operations.

3 . List two differences between microprogrammed and hardwired control unit

Aspect	Hardwired Control Unit	Microprogrammed Control Unit
Speed	Faster, since control signals are generated directly using hardware circuits.	Slower, since control signals are fetched from control memory (microinstructions).
Flexibility	Inflexible – difficult to modify; requires hardware changes.	Flexible – easy to modify or update by changing microprograms in control memory.

4 . **Describe Address Sequencing.**

Address Sequencing is the process of determining the sequence of memory addresses from which instructions or data are fetched during program execution. It controls how the CPU moves through instructions, whether sequentially, via jumps, branches, or interrupts, ensuring the correct flow of program control.

5 . Explain RISC.

RISC (Reduced Instruction Set Computer) is a CPU design that uses a small, simple set of instructions, each executing quickly (usually in one cycle), to improve performance and simplify hardware.

6 . Explain CISC.

CISC (Complex Instruction Set Computer) is a CPU design that uses a large set of complex instructions, some of which can perform multiple tasks in a single instruction, aiming to reduce the number of instructions per program.

7 . List the two differences between RISC and CISC

Aspect	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instruction Set	Uses a **small and simple set** of instructions; each instruction executes in a single clock cycle.	Uses a **large and complex set** of instructions; some take multiple clock cycles.
Execution Speed	Faster due to simpler instructions and pipelining.	Slower compared to RISC because of complex instructions and variable execution time.

8 .Define Addressing mode.

Addressing mode defines the way an instruction specifies the location or the method to access its operands (data). It determines how the CPU interprets the address field in an instruction to fetch the actual data needed for processing.

9 . Describe with an example for direct addressing and indirect addressing mode.

- Direct Addressing: The instruction contains the actual address of the operand. *Example:* LOAD R1, 100 — Load data from memory location 100 into R1.
- Indirect Addressing: The instruction contains the address of a memory location that holds the operand's address. *Example:* LOAD R1, (100) — Memory location 100 holds the address of the actual data to load into R1.

10 . List the types of Instruction format.

Here are the common types of instruction formats:

- Zero-address format

- One-address format
- Two-address format
- Three-address format.

11 . List few data transfer Instructions

Here are a few common Data Transfer Instructions:

- MOV – Move data from one location to another
- LOAD – Load data from memory to a register
- STORE – Store data from a register to memory
- PUSH – Place data onto the stack
- POP – Remove data from the stack
- XCHG – Exchange data between registers or between register and memory.

12 . List few data manipulation Instructions.

Here are a few common Data Manipulation Instructions:

- ADD – Add two values
- SUB – Subtract one value from another
- MUL – Multiply two values
- DIV – Divide one value by another
- INC – Increment a value by 1
- DEC – Decrement a value by 1
- NEG – Negate a value (change its sign).

13 . Differentiate between a "JUMP" and a "CALL" instruction in terms of program control.

JUMP Instruction	CALL Instruction
Transfers control to another part of the program but does **not save return address**.	Transfers control to a subroutine and **saves return address** for returning.
Used for **branching/loops**.	Used for **subroutine/procedure calls**.

14 . Describe microprogram.

A microprogram is a set of low-level instructions (called microinstructions) stored in control memory that define how the control unit of a CPU executes machine-level instructions. It breaks down complex instructions into simpler steps to control hardware operations like data transfers and ALU actions.

15 . Explain control memory.

Control memory is a special memory in a CPU that stores microinstructions used by the control unit to generate control signals for executing machine instructions.

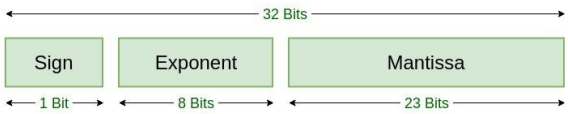
Module 3: Data Representation and Computer Arithmetic

1 . List the two types of data representations.

The two main types of data representations are:

- Numeric Data Representation – Used to represent numbers (e.g., integers, floating-point).
- Non-Numeric Data Representation – Used to represent characters and symbols (e.g., ASCII, Unicode).

2 . Draw the IEEE floating point representation.



Single Precision
IEEE 754 Floating-Point Standard

3 . Explain 2's complement representation.

2's Complement Representation is a method used to represent signed integers in binary form, allowing easy addition and subtraction of positive and negative numbers.

Example (4-bit system):

- +5 = 0101
- To find -5:

1. Take binary of 5 → 0101
2. Invert bits → 1010
3. Add 1 → 1011 (This is -5 in 2's complement)

4 . Describe the three techniques to represent signed numbers

- Sign-Magnitude: Uses the MSB as sign (0 = positive, 1 = negative), rest for magnitude.
- 1's Complement: Negative numbers are the bitwise inverse of positive numbers; has two zeros.
- 2's Complement: Negative numbers are 1's complement plus 1; allows simple arithmetic and has one zero.

5 . Describe 9's complement.

9's Complement is a method used to represent and perform subtraction on decimal numbers. To find the 9's complement of a decimal number, you subtract each digit from 9.

Example:Number:4729

9's Complement:

9−4=5 9−7=2 9−2=7 9−9=0

So, 9's complement of 4729 is **5270**.

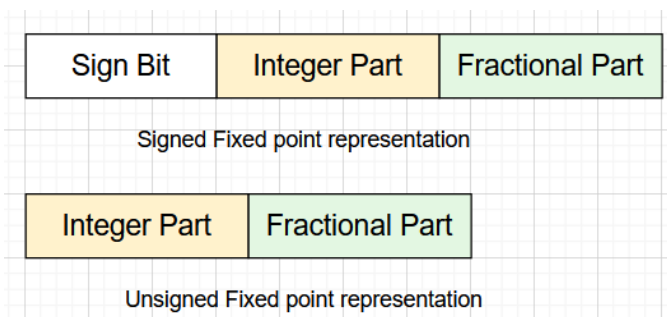
6 . Describe Decimal Arithmetic Unit.

A Decimal Arithmetic Unit (DAU) is a specialized hardware component or a set of circuits designed to perform arithmetic operations directly on decimal numbers (Binary Coded Decimal or BCD) rather than converting them to binary for computation.

7 . List the difference between restoring and non-restoring division algorithm

Restoring Division	Non-Restoring Division
Performs subtraction; if result is negative, the divisor is added back (restored).	Performs subtraction or addition depending on the sign, without restoring immediately.
Slower due to extra restoring step.	Faster since restoring step is avoided.

8. Draw the fixed- point representation



9 . What are the decimal arithmetic operations?

TABLE 10-5 Decimal Arithmetic Microoperation Symbols

Symbolic Designation	Description
$A \leftarrow A + B$	Add decimal numbers and transfer sum into A
$\overline{B}$	9's complement of B
$A \leftarrow A + \overline{B} + 1$	Content of A plus 10's complement of B into A
$Q_L \leftarrow Q_L + 1$	Increment BCD number in Q_L
dshr A	Decimal shift-right register A
dshl A	Decimal shift-left register A

10 . Explain BCD addition with an example.

BCD Addition is the process of adding two numbers represented in Binary-Coded Decimal (BCD) format. Since each digit is stored in 4 bits (0–9), special rules apply to correct results that go beyond 9 in any digit.

Example: Add 59 and 27 in BCD

- 59 in BCD: 0101 1001
- 27 in BCD: 0010 0111

Result:

1000 0110 → 86 decimal

MODULE 4 : Input-Output Organization and Memory Organization

1 . Define Interrupt.

An interrupt is a signal that temporarily pauses the CPU's current tasks to handle a high-priority event, after which the CPU resumes the original task.

2 . Explain Direct Memory Access

Direct Memory Access (DMA) is a feature that allows certain hardware devices to transfer data directly to or from the main memory without involving the CPU. This speeds up data transfer and frees the CPU to perform other tasks while the transfer happens independently.

3 . Describe the types of ROM

- Masked ROM (MROM):
- Programmed during manufacturing; contents are permanent and cannot be changed.
- Programmable ROM (PROM):
- Can be programmed once after manufacturing using a special device called a programmer.
- Erasable Programmable ROM (EPROM):
- Can be erased by exposing to UV light and reprogrammed multiple times.
- Electrically Erasable Programmable ROM (EEPROM):
- Can be electrically erased and reprogrammed without removing it from the system.

- Flash Memory:

- A type of EEPROM that can erase and write data in blocks quickly; widely used in modern devices.

4 . Describe the types of RAM.

- Static RAM (SRAM):

- Uses flip-flops to store each bit.
- Faster and more expensive.
- Does not need to be refreshed.
- Used in cache memory.

- Dynamic RAM (DRAM):

- Stores bits as charges in capacitors.
- Slower and cheaper.
- Needs periodic refreshing to retain data.
- Used as main system memory.

- Synchronous DRAM (SDRAM):

- A type of DRAM synchronized with the system clock.
- Faster than conventional DRAM.

- Double Data Rate SDRAM (DDR SDRAM):

- Transfers data on both rising and falling clock edges, doubling bandwidth over SDRAM.

5 . Define Cache memory.

Cache memory is a small, fast memory located close to the CPU that stores frequently accessed data and instructions to speed up processing by reducing the time needed to access data from the main memory.

6 . Draw the Memory Hierarchy

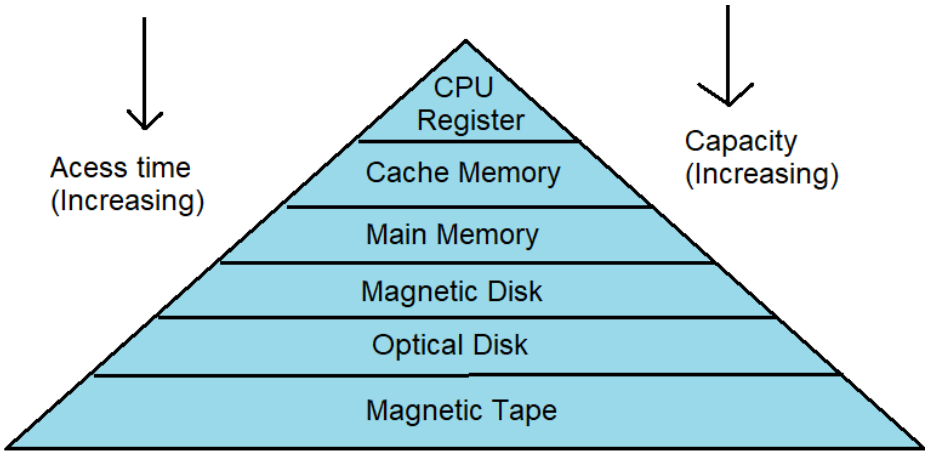


Fig:- Memory Hierarchy

7 . Describe the Secondary Memory with examples.

Secondary Memory is a type of non-volatile storage used to store data and programs permanently. Unlike primary memory (RAM), it retains data even when the computer is turned off. It typically has larger capacity but slower access speed.

Examples of Secondary Memory:

- Hard Disk Drives (HDDs)
- Solid State Drives (SSDs)
- Optical Discs (CDs, DVDs)
- Magnetic Tapes
- Flash Drives / USB Drives

8 . Explain Cache Hit and cache Miss.

* Cache Hit: Occurs when the CPU requests data or an instruction, and it is found in the cache memory. This results in a fast access. * Cache Miss: Occurs when the CPU requests data or an instruction, and it is *not* found in the cache. The CPU then has to fetch the data from a slower memory level (like main memory), which takes more time.

9 . Define Priority Interrupt.

Priority Interrupt is a type of interrupt system where multiple interrupt requests are assigned different priority levels. When several interrupts occur simultaneously, the CPU responds first to the highest priority interrupt, ensuring critical tasks are handled before less urgent ones.

10 . Explain Daisy chaining.

Daisy Chaining is a technique used to manage multiple interrupt requests in a system by connecting interrupt devices in series (like a chain). When an interrupt occurs, the CPU sends an acknowledge signal down the chain until it reaches the device with the highest priority that requested the interrupt. This device then takes control, ensuring prioritized handling of interrupts.

MODULE 5: Reduced Instruction Set Computer, Pipeline and Vector Processing, Multi Processors

1 . Explain Pipelining.

Pipelining is a technique used in CPUs to improve instruction throughput by overlapping the execution of multiple instructions. The instruction processing is divided into stages (like fetch, decode, execute), and while one instruction is being executed in one stage, others are processed in different stages simultaneously—much like an assembly line. This increases overall performance by allowing multiple instructions to be worked on at the same time.

2 . Explain Parallel Processing

Parallel Processing is a computing technique where multiple processors or cores execute multiple tasks or instructions simultaneously. It divides a large problem into smaller sub-tasks that run at the same time, significantly speeding up processing and improving performance for complex or large-scale computations.

3 . Define RISC.

RISC (Reduced Instruction Set Computer) is a CPU design that uses a small, simple set of instructions, each executing quickly (usually in one cycle), to improve performance and simplify hardware.

4 . Define CISC

CISC (Complex Instruction Set Computer) is a type of CPU design that uses a large set of complex instructions, some capable of performing multiple low-level operations within a single instruction, aiming to reduce the number of instructions per program.

5 . Define a multiprocessor system

A multiprocessor system is a computer system that has two or more processors (CPUs) working together to perform tasks simultaneously, improving performance, reliability, and throughput by sharing the workload.

6 . List the Flynn's Classification of computers.

Flynn's Classification of computers divides them into four types based on instruction and data streams:

- SISD (Single Instruction, Single Data)
- SIMD (Single Instruction, Multiple Data)
- MISD (Multiple Instruction, Single Data)
- MIMD (Multiple Instruction, Multiple Data)

7 . Explain Cache Coherence.

Cache Coherence refers to the consistency of data stored in multiple caches of a multiprocessor system. When multiple processors have their own caches, cache coherence ensures that any changes made to data in one cache are reflected in all other caches holding that data, preventing processors from working with outdated or inconsistent information.

8 . Define Array Processor.

An Array Processor is a type of parallel processor designed to perform the same operation simultaneously on multiple data elements arranged in arrays or vectors. It's especially efficient for tasks involving large-scale mathematical computations, such as in scientific and engineering applications.

9 . List the types of Pipelining.

- Arithmetic Pipelining – Used for arithmetic operations divided into stages.

- Instruction Pipelining – Overlaps different stages of instruction execution (fetch, decode, execute, etc.).

- Graphics Pipelining – Processes graphics tasks in stages for faster rendering.

- RISC Pipeline – A fixed pipeline used in RISC processors with uniform instruction stages.

- Superpipelining – Divides pipeline stages into smaller sub-stages to increase instruction throughput.

10 . Explain Vector Processing.

Vector Processing is a computing technique where a single instruction operates simultaneously on multiple data elements arranged in vectors (one-dimensional arrays). It's highly efficient for tasks involving large-scale mathematical operations like scientific simulations, graphics, and signal processing, as it processes data in parallel, reducing execution time compared to scalar processing (which handles one data element at a time).